

Lec-02: Linear Search

Definition

- Linear Search is used to find a specific element in a unsorted list or unsorted array.
- It checks each element one by one from the beginning until the required element is found or the end of the list is reached.

How Linear Search Works

Suppose we have the following list:

[15, 8, 23, 41, 12]

We want to search for the element **41**.

Step	Current Element	Comparison	Result
1	15	15 == 41	No
2	8	8 == 41	No
3	23	23 == 41	No
4	41	41 == 41	Yes (Found)

The search stops immediately after finding the required element.

Algorithm

1. Start from the first element.
2. Compare the current element with the target element.
3. If both are equal, return the index of the element.
4. Otherwise, move to the next element.
5. Repeat until the element is found or the end of the list is reached.
6. If the element is not found, return -1.

Python Program

```
1 def linear_search(arr, target):
2     for i in range(len(arr)):
3         if arr[i] == target:
4             return i
5     return -1
6
7 numbers = [15, 8, 23, 41, 12]
8 key = 41
9
10 result = linear_search(numbers, key)
11
12 if result != -1:
13     print("Element found at index:", result)
14 else:
15     print("Element not found")
```

Output

Element found at index: 3

Example

Search for **20** in the following list:

[5, 10, 15, 25, 30]

Comparisons performed:

- $5 == 20 \rightarrow \text{No}$
- $10 == 20 \rightarrow \text{No}$
- $15 == 20 \rightarrow \text{No}$
- $25 == 20 \rightarrow \text{No}$
- $30 == 20 \rightarrow \text{No}$

Since all elements have been checked and none match the target, the algorithm returns -1.

Time Complexity

Case	Complexity	Reason
Best Case	$O(1)$	Element is the first item.
Average Case	$O(n)$	Element is somewhere in the middle.
Worst Case	$O(n)$	Element is the last item or not present.

Space Complexity

$O(1)$

Linear search uses only a few variables regardless of the input size.

Advantages

- Simple and easy to understand.
- Easy to implement.
- Works with both sorted and unsorted data.
- Requires no additional memory.

Disadvantages

- Inefficient for large datasets.
- May need to examine every element.
- Slower than Binary Search for large sorted datasets.

Applications

- Mainly used to search for elements in unsorted arrays or lists.
- Suitable for small data sets.

Summary

- Linear search checks elements one by one.
- The search stops immediately when the target element is found.
- If the target is not present, it returns -1 .
- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

Relation to Python's `find()` Method

The Python string method `find()` searches for a substring by scanning the string from left to right.

```
1 text = "Data Structures"
2
3 print(text.find("Str"))
```

Output:5

Important Note

From a conceptual point of view, the Python `find()` method performs a **linear search**. It examines the characters from left to right until the required substring is found or the end of the string is reached.

Although Python internally uses highly optimized string-search algorithms, beginners can understand `find()` as following the same basic idea as linear search.